Agentic AI Builder

Demo capability expansion | Implementation scope after current UI and backend fixes

---

Goal. Extend the existing product rather than rebuild it. Make external connections genuine, use prepared analysis assets for the expensive intelligence layer, and connect everything through the current workflow builder and monitoring screens.

1. Google account and application connections

- **One-click connection.** Add Connect Google using Authlib/OAuth 2.0. The user signs in and grants selected scopes; the backend handles callback, token storage, refresh, reconnection and revocation.
- **Usable tools.** Expose Gmail read/search/draft/send, Calendar list/create/update/delete events, and Sheets read/create/update operations as authorized backend tools.
- **Simple UI.** Display connected account, permitted capabilities, connection health and disconnect. Use demo/test Google accounts and explicitly granted permissions.

2. Real MongoDB and Redis cloud connections

- **Connection manager.** Add MongoDB via PyMongo and Redis via redis-py inside the existing integrations architecture. Accept server-side connection settings; test connectivity and save encrypted credentials.
- **Source discovery.** For MongoDB, list databases, collections, sampled fields and document counts. For Redis, inspect bounded key samples, key types and TTL metadata using SCAN rather than KEYS.
- **Safety.** Default to read-only access, restrict connection targets and timeouts, limit samples, and isolate credentials by workspace. Show failures instead of fake connection success.

3. Single Data Intelligence workspace menu

- **Data Sources.** Show connected databases, collection inventories, refresh status and source metadata.
- **Knowledge Graph.** Explore sources, collections, fields and documented links in an interactive node graph.
- **EDA.** Display prepared dataset profiles, statistical summaries and charts.
- **AI Insights.** Show saved findings, source evidence and questions the agent can answer.

4. Visible knowledge graph expansion on new connection

- **Automatic update.** After a successful source connection, scan metadata and merge source/collection/field nodes into a saved graph JSON through stable IDs. Preserve existing graph layout and edges.
- **Visual feedback.** Animate newly added nodes and links, highlight the new source, and show counts of new collections and fields. Provide a Refresh graph action.
- **Relationship quality.** Use metadata-derived links automatically; load prepared cross-source business relationships only for the known demo datasets. Do not imply that all joins or semantic relationships were discovered live.

Implementation scope (continued)

5. Prepared EDA, dashboards and knowledge assets

- **Offline analysis.** Use Python/Pandas to profile selected demo datasets once; save versioned JSON for columns, nulls, duplicates, distributions, descriptive statistics, trends, correlations and insight summaries.
- **High-impact visuals.** Render KPI cards, time-series trends, bar charts, distributions, correlation heatmaps, rankings, and entity/relationship graph views with existing frontend chart components.
- **Dataset matching.** Bind each prepared EDA report and knowledge graph to a particular source/dataset ID and snapshot timestamp. New arbitrary databases receive real metadata discovery without unrelated precomputed analysis.
- **Lightweight lake.** Keep selected CSV/Parquet snapshots and metadata in a simple local or object-storage structure; a separate lakehouse platform is not required for this demo.

6. AI access and workflow generation

- **Agent tools.** Expose backend functions such as list_data_sources, get_dataset_profile, get_eda_insights, get_knowledge_graph, find_related_entities and get_chart_data. Return grounded data and freshness details.
- **Workflow composition.** Extend the existing planner with the available data-source and Google-tool catalog. Use a few validated workflow templates that can be edited, simulated and executed through existing contracts.
- **Real versus preview actions.** Run genuinely supported Google and database operations through the backend. Keep illustrative or unsupported actions in a clearly marked simulation mode; preserve permission checks and approvals.

7. Detailed workflow visuals and investor-ready interaction

- **Builder.** Show trigger, data access, analysis lookup, agent reasoning, decisions, approvals and output actions as separate editable workflow nodes. Use readable branches and fit-to-screen layout.
- **Execution effect.** Animate the active path; highlight running/completed/failed nodes; display tool names, step durations, evidence used, status messages, safe inputs/outputs and final artifacts.
- **Repeatability.** Use fixed demo datasets, test Google accounts, controlled recipients, stable templates and a Reset demo operation that restores saved fixtures without deleting real connected-account data.
- **Error credibility.** Include proper loading, permission, retry, connection-failure and empty states. Never present simulated events as live API results.

8. Implementation boundaries and completion checks

Reuse: existing Next.js navigation, FastAPI integrations, React Flow builder, workflow lifecycle, run monitoring and permission model. **Do not add:** Neo4j, distributed query infrastructure, general-purpose live EDA, cross-database federation, or a new orchestration engine.

Verify: OAuth callback and token refresh; Gmail/Calendar/Sheets authorized actions; MongoDB/Redis connection and bounded discovery; visible KG delta after connecting a database; prepared charts mapped to the right dataset; agent retrieval of the saved evidence; successful workflow simulation and controlled live execution; repeatable demo reset.